



O&C: Perspectives for Submission Infrastructure

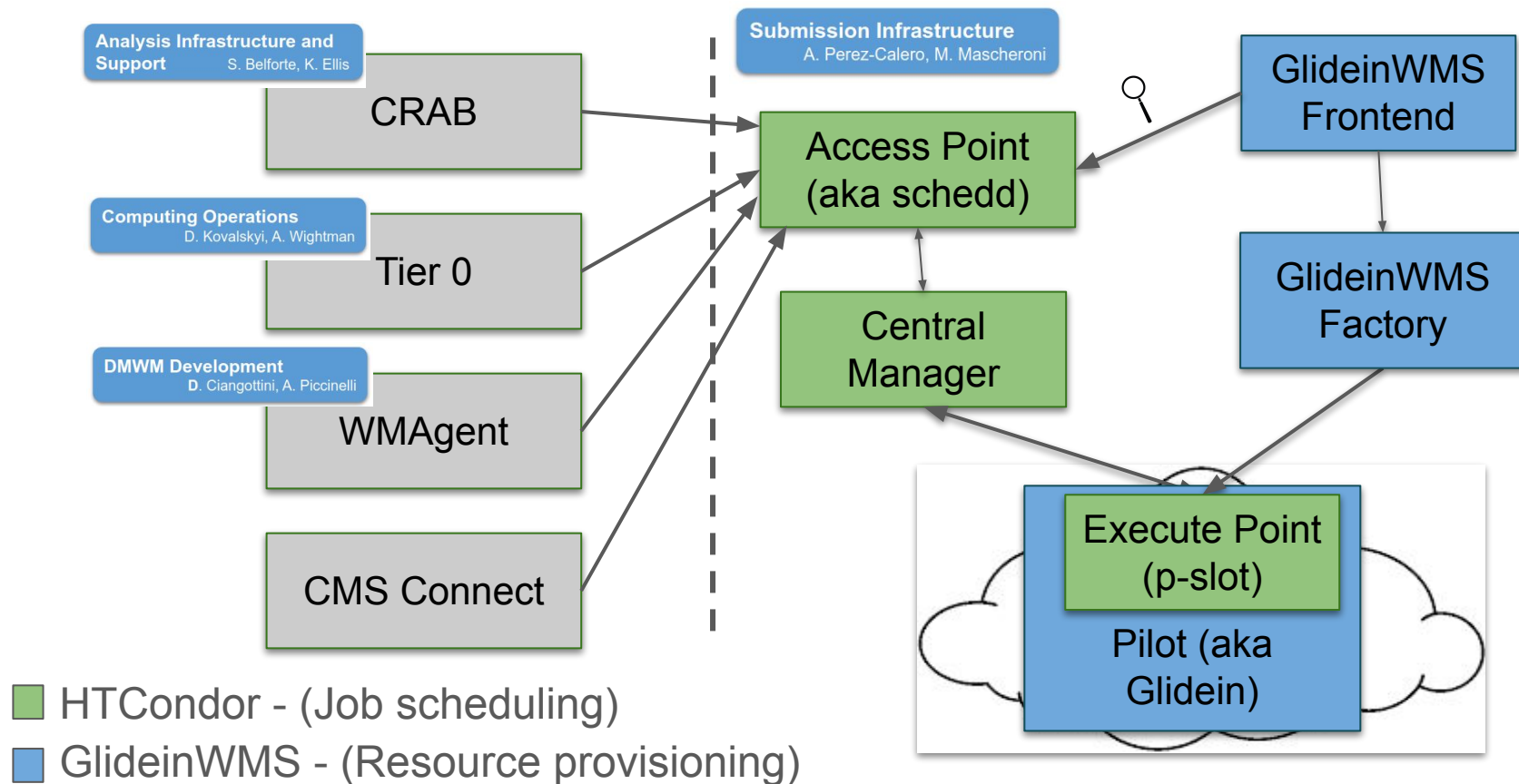
Marco Mascheroni, A. Pérez-Calero Yzquierdo
On behalf of the Submission Infrastructure team

CMS DMWM Workshop, CERN, Sept. 17th 2025

Submission Infrastructure in WM context

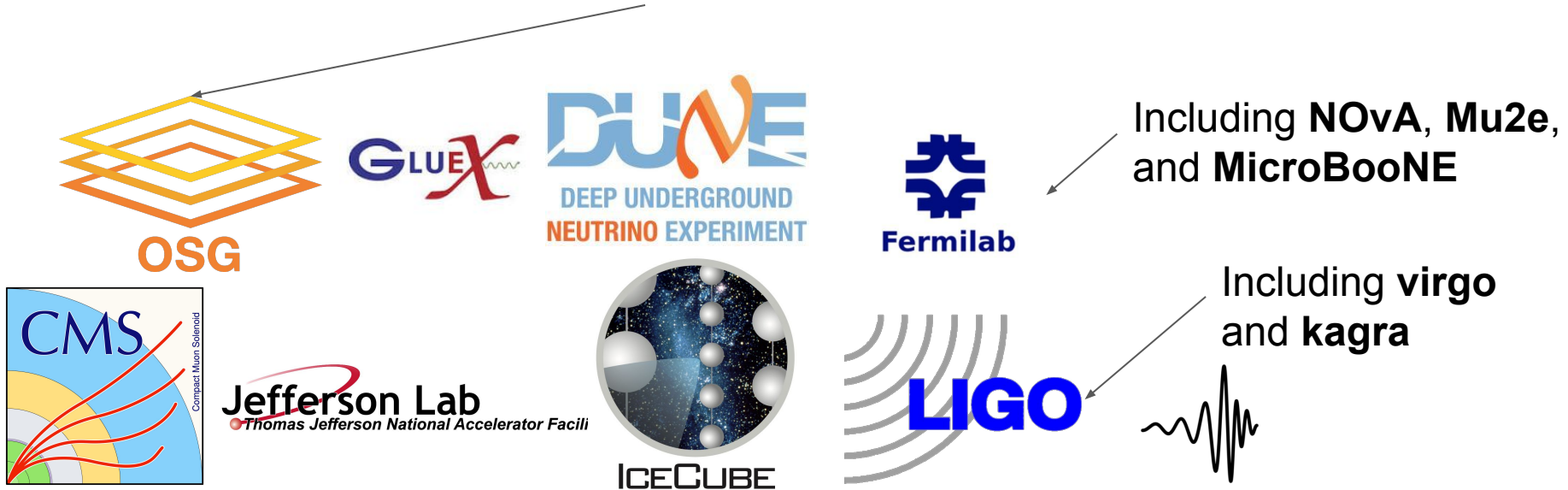
Workflows

Jobs (SI)



Tools used

- **GlideinWMS** is used by >20 experiments without constraining development.
 - On the OSG side alone about a **dozen different experiments** and up to about **100 individual research groups** across all of science use glideinWMS.





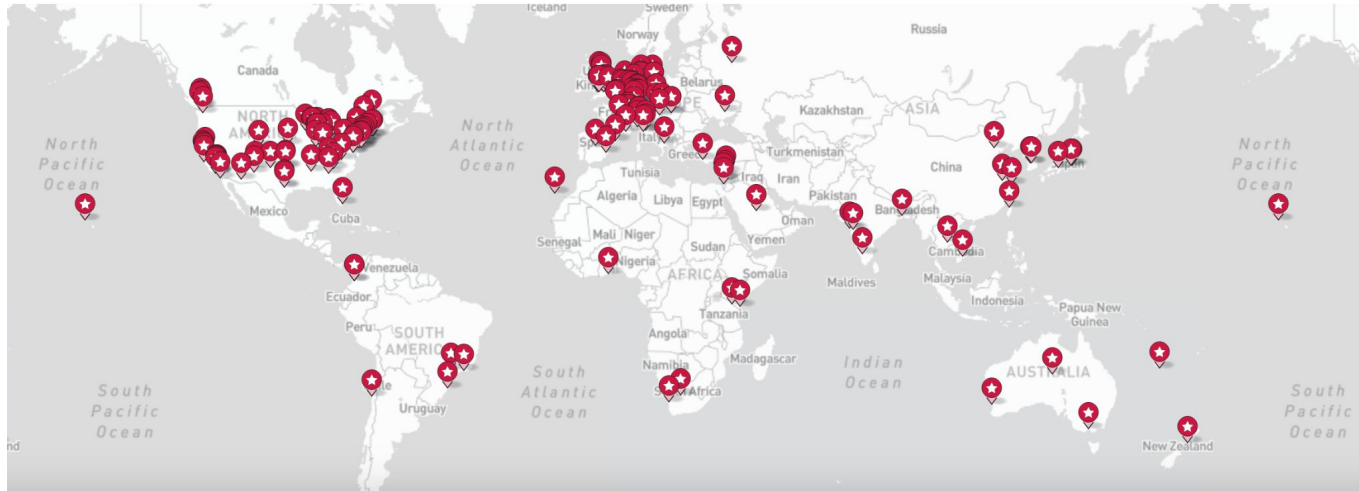
A global solution for WM

HTCSS - Trusted to Manage the Workloads of 100s of Academic, Government and Industry Institutions

Map of 175 Pools Utilizing HTCSS (user reported)

Add your organization to help us report our impact!

Add Organization +



Resource Provisioning

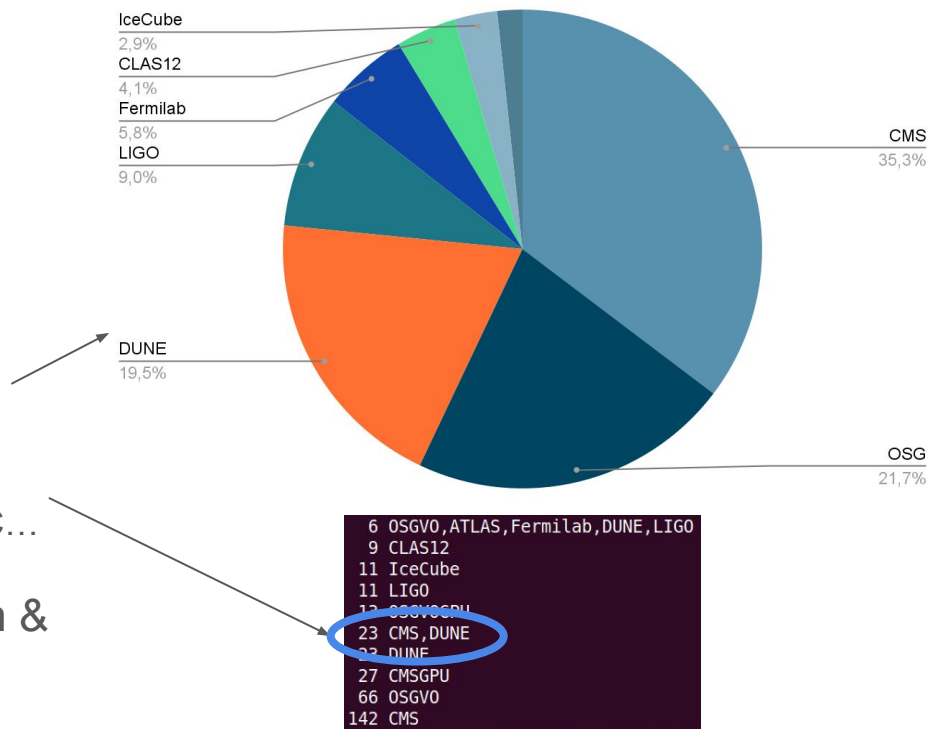
SI responsibilities: (1) Factory operations

- Factory operations at a glance
 - Maintain and update the **catalogue** of CMS batch system queues (**factory entries**) and their configuration
 - Entry = CE + submit attributes (e.g. queue) + CMS preference for utilization (Num cores, Memory)
 - Interact with site admins for:
 - **Commissioning** new sites
 - Authentication and **CE-related issues**
 - Pilot **execution problems** (validation errors, container issues, etc.)
- Effort is **shared across VOs**, especially:
 - Testing, validation, and software maintenance
 - Site commissioning and support (up to a certain extent)

Shared Factories

CMS uses both dedicated and shared glideinWMS factories, taking advantage of **collaboration with other experiments**.

- 4 glideinWMS factories
 - **CERN** and **FNAL**: CMS Only
 - **Tiger**: CMS + OSG + Other experiments
 - **Tempest**: OSG + Other experiments
- ~500 active entries in the OSG factory
 - 208 CMS, 128 OSG, 115 DUNE, 53 LIGO
 - 34 Fermilab, 24 CLAS12, 17 IceCube, 10 EIC...
 - Many entries are shared across experiments
- Advantage: **shared software validation & testing**



SI responsibilities: (2) Develop and Maintain CMS specific node validation scripts

Common (factory-shared) **scripts**

- OS validation
- Select & download HTCondor tarball (OS/arch)
- Setup credentials for collector callback
- CVMFS setup (via cvmfsexec, “Deuclion”)

CMS-specific (frontend) **scripts**

- Squid validation
- Siteconf validation
- Benchmark (DIRAC)
- Microarchitecture validation
- Site customization:
 - Custom classads
 - Custom start expressions

Jobs Scheduling



SI responsibilities: (3) Global pool

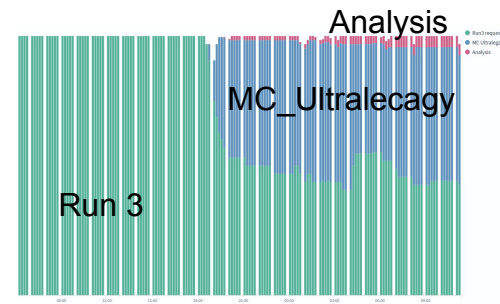
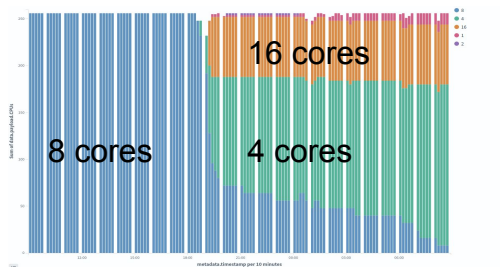
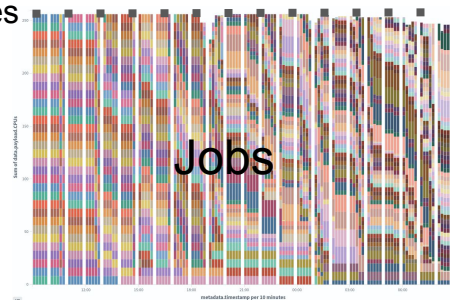
Manage Global pool and make sure priorities and shares are respected always keeping an eye on (scheduling) efficiency

Different aspects of scheduling a whole node pilot at FNAL
(same pilot job in the 4 images)

Scheduling inefficiencies are minimal



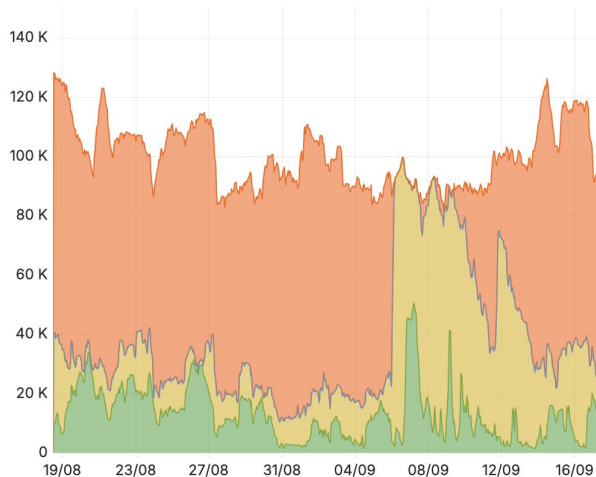
256 cores



Job Scheduling

- Pilot-based late binding
 - Resources claimed first, jobs matched later
- Global fair share & priorities
 - Unified pool across CMS activities
- Quotas and priorities
 - High priority can take 100%

Running cores ⓘ



Name	Mean	Last *	Max	Min	Total
analysis	13.2 K	15.2 K	50.6 K	0	9.47 Mil
production	24.3 K	9.54 K	96.9 K	0	17.5 Mil
test	7.35	7.40	131	0	5.29 K
tier0	62.3 K	68.1 K	98.4 K	0	44.9 Mil
TOTAL	99.7 K	92.9 K	128 K	0	71.8 Mil

	Tier 0 Jobs	Production Jobs	Analysis Jobs
CERN	80%	10%	10%
Tier 1s	60%	38%	2%
US Tier 2	/	75%	25%
Non US Tier 2	/	60%	40%

SI Perspective

Strategies for Optimal Utilization of Resources

SI does not define requirements, but **provides guidelines** and mechanisms for most efficient use of resources.

- **Materialize **job** in schedulers as soon as possible**
 - E.g.: Pilots retire after ~4h, but often fragmented (jobs request 8–16h). Short, low-priority jobs could fill expiring pilots if they were in schedulers
 - Same logic applies to memory and other resources we schedule against
- **Maintain **steady pool pressure****
 - The pool has a certain inertia, so any sudden change takes time to readjust and causes inefficiency
 - Example: analysis overshooting its share, or Tier-0 draining single-core repacks
- **Balance **high-memory** workloads**
 - High memory jobs can impact pool utilization if the average mem/core surpasses the typical 2GB/core
 - Acceptable if monitored and understood
 - Other VOs push this inefficiency to sites; SI handles it within CMS
- Canceling workflows should be painless
- Job logs should be easily available

Functionalities to Leverage in the Next System (1)

- Technically available, but not in our (conservative) policies to use:
 - **Preemption** and forced draining or **defragmentation** of a resource
 - **Checkpointing** of jobs
 - Per payload/pilot **cgroups**
 - Use **subgroup shares**
 - Production share can be further broken down by physic group for example

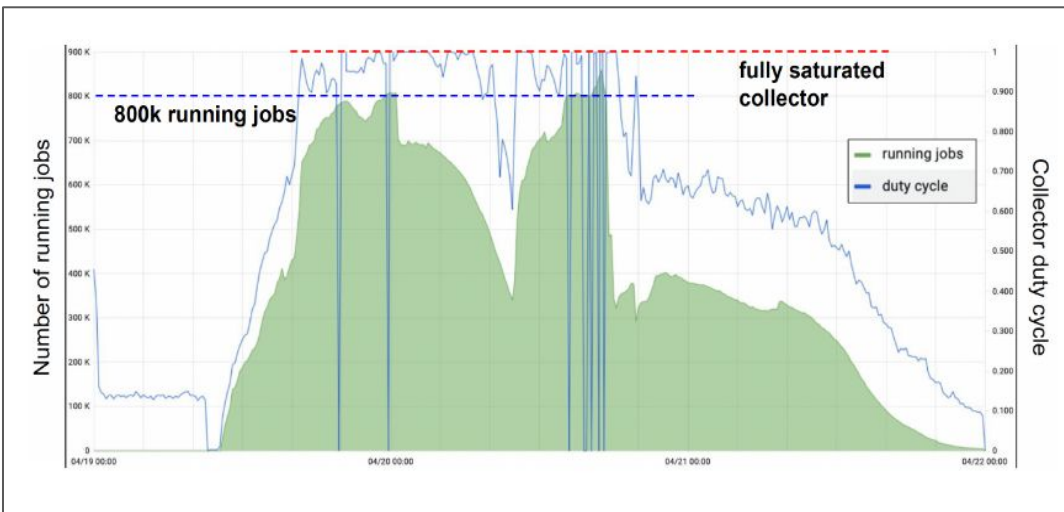
Functionalities to Leverage in the Next System (2)

- Job Router flexibility
 - **Dynamically adjust job requirements** (e.g., block-to-site conversion)
 - E.g.: Dynamic blacklisting of problematic sites
- **Decouple workflow materialization** into jobs (i.e current task of WMAgent) **from the scheduler** agents that manage jobs into the condor pools (schedds)
 - Present one-to-one tight coupling introduces undesired dependencies between the two components, limiting scalability and affecting resource utilization
- Workflow orchestration with **DAGMan**
 - Native support for dependencies and retry logic
 - See next slide for details

Prototyping the new system

We in SI have quite some experience in regularly stressing our infrastructure to preventively find the future limitations on scalability.

We believe potential future WM models should be fully stress-tested, including SI. We'd be of course willing and happy to participate and contribute.



Last iteration of the test in **2023** at **800k** running jobs

Should we also investigate scalability in terms of idle jobs?

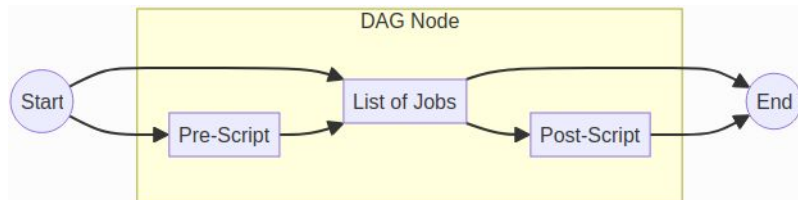
DAGMan

HTCondor is not only about job scheduling:
DAGMan enables complex workflow management

- DAGMan (Directed Acyclic Graph Manager): orchestrates jobs with dependencies
- Express workflows as a graph: parents → children → grandchildren
 - Automatic retry and error recovery
- Enables “workflow as a single job” abstraction

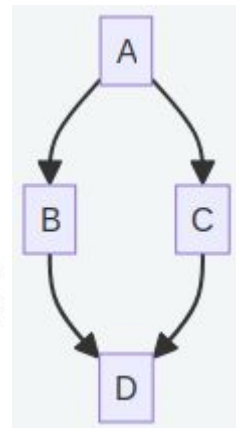
With DAGMan, CMS can use HTCondor not only to run jobs, but to manage **multi-step workflows**.

Used by CRAB, LIGO collaboration and LSST/Rubin



File name: diamond.dag

```
JOB A A.sub
JOB B B.sub
JOB C C.sub
JOB D D.sub
SCRIPT PRE B pre.sh $NODE .gz
SCRIPT PRE C pre.sh $NODE .gz
PARENT A CHILD B C
PARENT B C CHILD D
```



Conclusions

- Scope of this talk
 - In this talk we focused on core operations and what we consider as **essential capacities** for the next CMS WM system
 - It was not our aim to discuss future extensions and **standalone evolution of SI** (overflow, overload, ioslots, gpus, arm, custom matchmaking, singularity, etc)
- SI perspective
 - **Keep core strengths** in our current model
 - Evaluate external tools (PanDA, DIRACx) especially for features not already covered by glideinWMS + HTCondor (e.g.: relationship with data management)
 - Adding new layers is possible but it comes with long term operational costs (see backup slides about modularity and discussion this morning)
 - Minimize unnecessary layers: the fewer, the better
- Principles going forward
 - Explore HTCondor features that are not yet implemented or fully exploited, e.g.
 - Advanced GPU scheduling and monitoring
 - DAGMan to dynamically build our workflows
 - Foster our collaboration with glideinWMS and HTCondor teams, considering they are also part of the CMS collaboration!
 - Eager to learn about our current and future challenges and tackle them together!

Backup slides

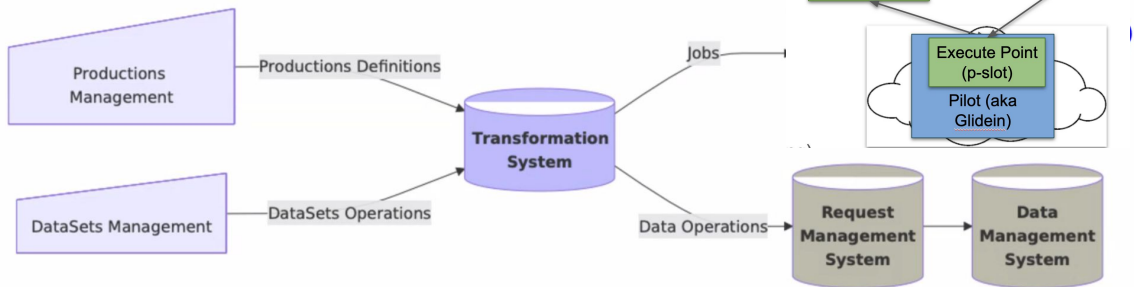
Comment on modularity

==> WM system must be modular with well defined interfaces so individual components can be easily replaced during the lifetime of the system (this will also allow parallel development and lower the bar for people to contribute)

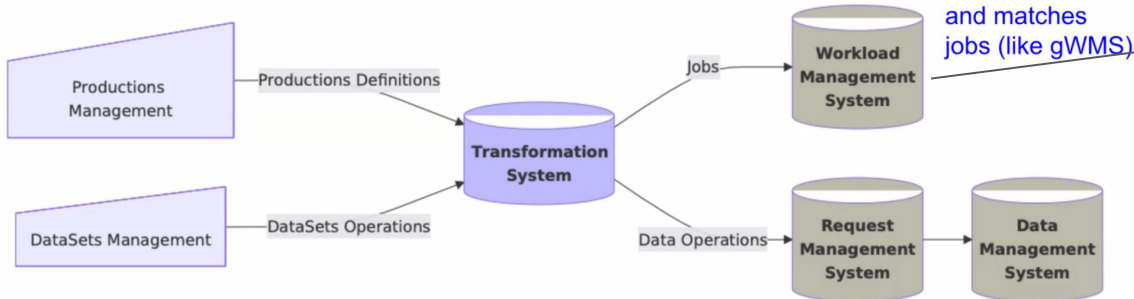
- Modularity is key also for external solutions
 - Prevents duplication of services
 - Allow to only adopt components that don't duplicate existing CMS functionality
 - E.g.: Other experiments already operate factories. With modular design, CMS can plug in or interoperate instead of duplicating effort

Replacing vs stacking

Defines the work

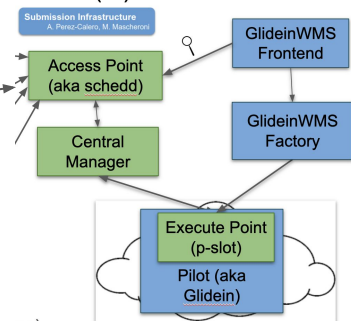


Defines the work



Moves data

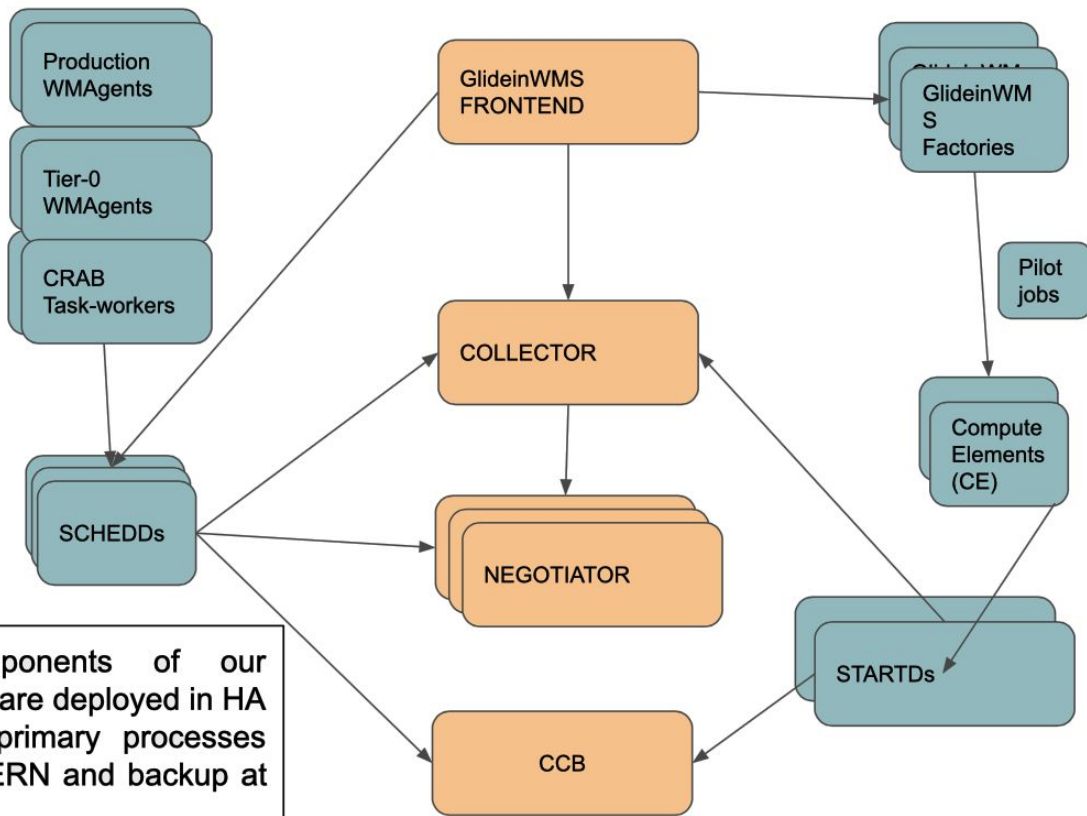
Replacing more development but less long term maintenance



Entry Definition

```
<entry name="CMSHTPC_T2_DE_DESY_grid-htc-ce03" auth_method="grid_proxy" comment="Entry created 2024-04-18 --Vaiva" enabled="True" gatekeeper="grid-htc-ce03.desy.de grid-htc-ce03.desy.de:9619"
<config>
  <max_jobs>
    <default_per_frontend glideins="50000" held="100" id="5000"/>
    <per_entry glideins="50000" held="1000" id="5000"/>
    <per_frontends>
    </per_frontends>
  </max_jobs>
  <release max_per_cycle="20" sleep="0.2"/>
  <remove max_per_cycle="5" sleep="0.2"/>
  <restrictions require_glidein_glexec_use="False" require_voms_proxy="False"/>
  <submit cluster_size="10" max_per_cycle="10" sleep="2" slots_layout="fixed">
    <submit_attrs>
      <submit_attr name="+maxMemory" value="20000"/>
      <submit_attr name="+maxWallTime" value="2880"/>
      <submit_attr name="+xcount" value="8"/>
    </submit_attrs>
  </submit>
</config>
<allow_frontends>
</allow_frontends>
<attrs>
  <attr name="GLEXEC_BIN" const="True" glidein_publish="False" job_publish="False" parameter="True" publish="True" type="string" value="NONE"/>
  <attr name="GLIDEIN_CMSSite" const="True" glidein_publish="True" job_publish="True" parameter="True" publish="True" type="string" value="T2_DE_DESY"/>
  <attr name="GLIDEIN_CPUS" const="True" glidein_publish="False" job_publish="True" parameter="True" publish="True" type="string" value="8"/>
  <attr name="GLIDEIN_Country" const="True" glidein_publish="True" job_publish="True" parameter="True" publish="True" type="string" value="DE"/>
  <attr name="GLIDEIN_MaxMemMBs" const="True" glidein_publish="True" job_publish="False" parameter="True" publish="True" type="int" value="20000"/>
  <attr name="GLIDEIN_Max_Walltime" const="True" glidein_publish="False" job_publish="False" parameter="True" publish="True" type="int" value="171000"/>
  <attr name="GLIDEIN_REQUIRED_OS" const="True" glidein_publish="True" job_publish="False" parameter="True" publish="True" type="string" value="any"/>
  <attr name="GLIDEIN_ResourceName" const="True" glidein_publish="True" job_publish="True" parameter="True" publish="True" type="string" value="DESY-HH"/>
  <attr name="GLIDEIN_Site" const="True" glidein_publish="True" job_publish="True" parameter="True" publish="True" type="string" value="DESY"/>
  <attr name="GLIDEIN_Supported_VOs" const="True" glidein_publish="False" job_publish="False" parameter="True" publish="True" type="string" value="CMS"/>
</attrs>
<files>
</files>
<infosys_refs>
</infosys_refs>
<monitorgroups>
</monitorgroups>
</entry>
```

CMS Global Pool and stability: **High-Availability deployment** and **redundancy and horizontal scaling of its components**



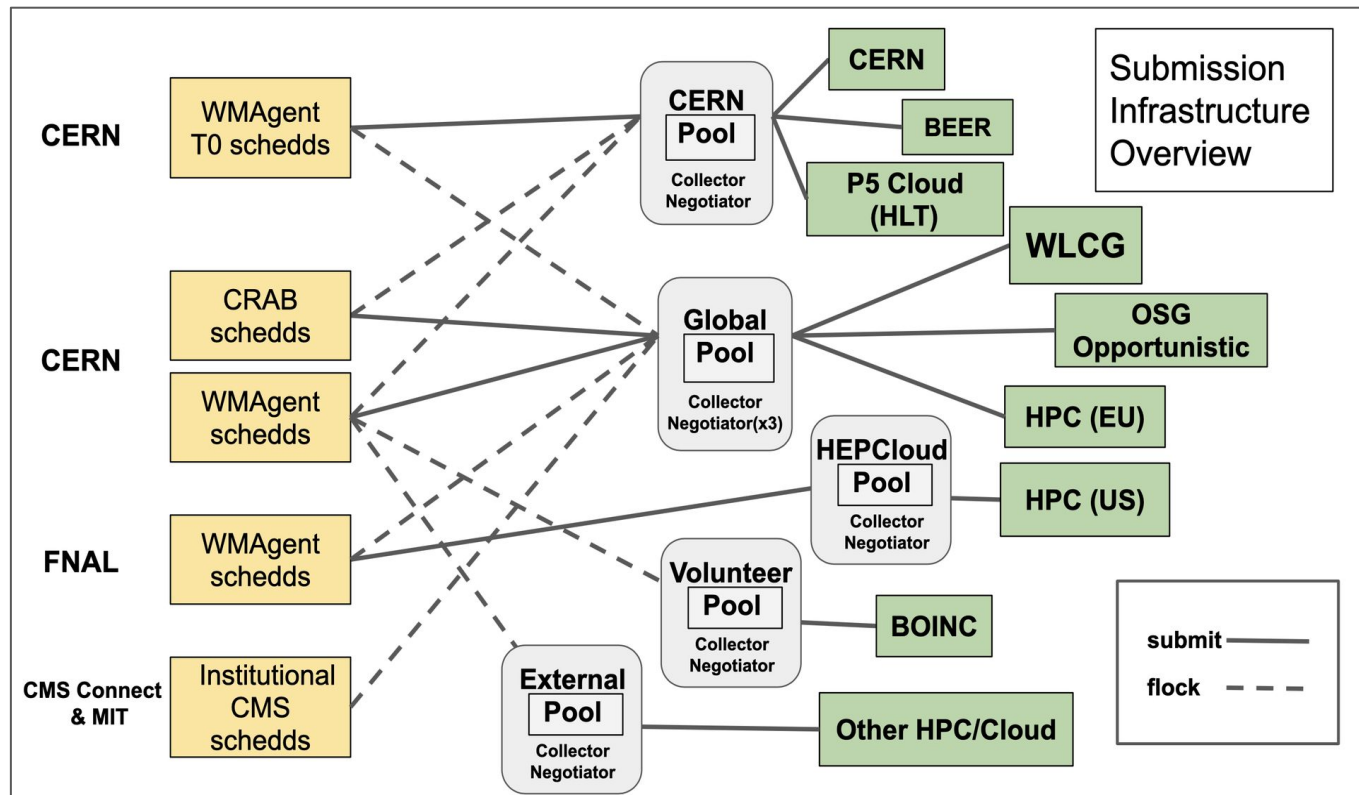
Critical components of our infrastructure are deployed in HA mode, with primary processes running at CERN and backup at FNAL

Pilot factories are deployed at CERN, FNAL and UCSD, overlapping with OSG's Factory Ops team

Schedulers are also deployed at CERN and FNAL, in numbers sufficient to manage CMS workloads for each major task category: centralized production (MC production and data processing campaigns), analysis tasks and Tier-0 jobs (including prompt data reconstruction at CERN).

CMS SI deployment with **multiple federated pools**

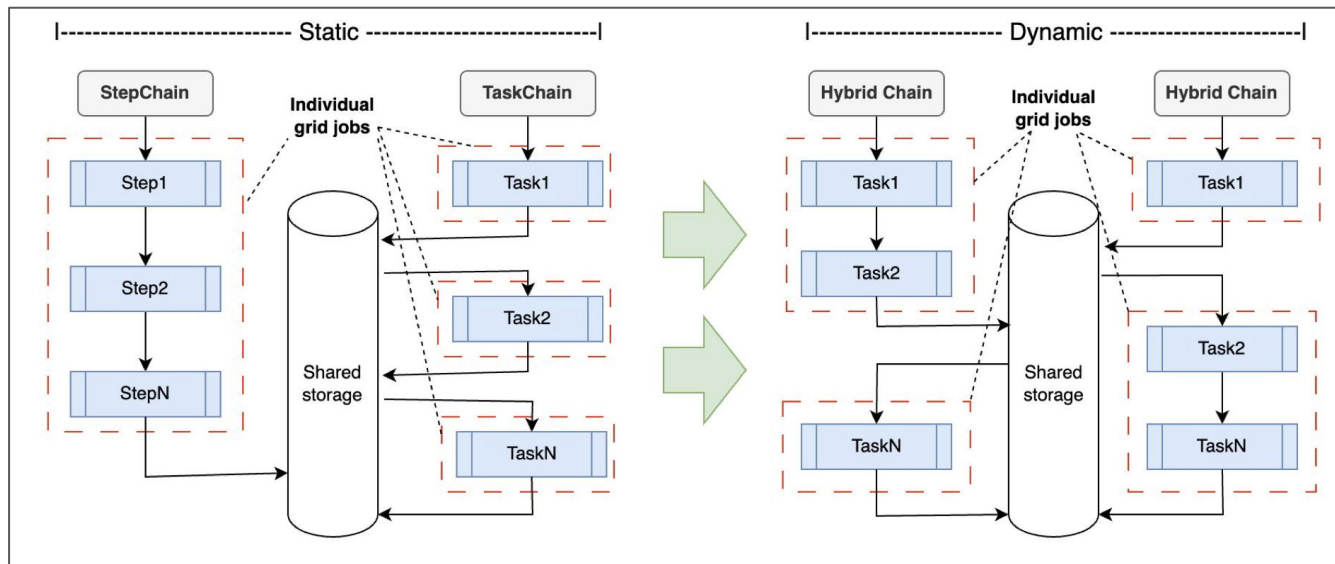
- Types of access point
- Types of execution point



Heterogeneous resources and tasks in the next CMS WM

- **Challenge:** how to **group interdependent tasks into jobs** that respect diverse constraints (e.g. CPU and GPU architecture) while **optimizing throughput** and overall resource utilization?
- **Critical** for the success of very large scale workloads in heterogeneous and limited resources

Static to Dynamic Workflow Composition



Just shown at the ongoing HTCondor WS

Heterogeneous resources and tasks in the next CMS WM

- **Strategy:** start from a **flexible workflow representation**, **grouping** similar tasks together, **evaluate** compatibility with constraints and trade-offs, then **suggest optimized workflow composition**

Workflow Composer Architecture

